



23. Python: Classes and Objects

Treat everything as objects

Previous Section:

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) 22. Python: Libraries](#)

Contents:

[1. Classes in Python](#)

[Main Components of a Class](#)

[2. Objects in Python](#)

[3. Object-Oriented Programming](#)

[Class Methods](#)

[Subclasses](#)

[Constructor and Destructor](#)

[Getter and Setter](#)

[Duck Typing](#)

[Some Other Concepts](#)

[Is treating everything as objects better than using many functions?](#)

[Summary](#)

[References](#)

When people first learn Python, they usually focus on functions, variables, loops, and conditions. And honestly, that makes sense. Those are the

foundations. But sooner or later, we start noticing something interesting: In Python, almost everything behaves like an object.

- Numbers? Objects.
- Strings? Objects.
- Lists? Objects.
- Even functions themselves? Also objects.

And this leads us to one of the biggest ideas in programming: **Object-Oriented Programming (OOP)**. Now, OOP is a huge topic on its own. Entire courses are built around it. So for now, we are not diving into advanced concepts yet. Instead, we focus on the core idea:

How do we create our own objects in Python?

Because sometimes, functions alone are not enough. We want to group data and behavior together into something meaningful.

For example:

- A `Student` object with a name, age, and major
- A `Car` object with speed and fuel
- A `BankAccount` object that can deposit and withdraw money

This is where **classes** come in. Think of a class as a blueprint. And the object? → The actual thing created from that blueprint.

So instead of programming with disconnected variables and functions, we begin organizing our code into structured entities that represent real-world ideas. And that's the beginning of Object-Oriented Programming.

1. Classes in Python

A **class** is a blueprint used to create objects. It defines: ***What data the object should store?*** and ***What actions the object can perform?***

In other words, a class combines both: **Attributes** → the data and **Methods** → the behaviors into a single structure.

Main Components of a Class

A Python class usually contains several important parts:

- **Class Name:** The name of the blueprint itself.

```
class Person:
```

By convention, class names usually start with a capital letter.

- **Initializer (Constructor):** This special method initializes the object when it is created.

```
def __init__(self):
```

This is commonly called the **constructor**. It automatically runs whenever we create a new object from the class.

- **Attributes:** Attributes are variables stored inside the object.

```
self.name = name  
self.age = age
```

These describe the properties of the object.

- **Methods:** Methods are functions that belong to a class.

```
def display(self):
```

They define what the object can do.

- **Objects:** An object is the actual instance created from the class.

```
student1 = Student("Alice", 20, "New York", "Computer Science")
```

Here, `student1` becomes an object created from the `Student` class.

- **Inheritance (Derived Classes):** One class can inherit features from another class. This is called **inheritance**. The original class is called the **base class**, while the new one is called the **derived class**. This allows code reuse and extension.

Example: Base Class and Derived Class

```
# Base class
class Person:
    def __init__(self, name, age, city):
        self.name = name
        self.age = age
        self.city = city

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}, City: {self.city}")

# Derived class
class Student(Person):
    def __init__(self, name, age, city, major):
        super().__init__(name, age, city)
        self.major = major

    def display(self):
        super().display()
        print(f"Major: {self.major}")

# Example usage
student1 = Student("Alice", 20, "New York", "Computer Science")
student1.display()
```

Name: Alice, Age: 20, City: New York
Major: Computer Science

In this example:

- `Person` is the base class
- `Student` inherits from `Person`
- `Student` reuses the attributes of `Person`
- `super()` allows the derived class to access methods from the parent class

So instead of rewriting everything from scratch, we extend existing functionality. And that is one of the biggest strengths of Object-Oriented Programming.

2. Objects in Python

Now that we understand what a class is, let's talk about the thing created from it: The **object**.

If a class is the blueprint, then the object is the actual product built from that blueprint. For example:

```
class Car:
    pass

car1 = Car()
```

Here:

- `Car` → the class
- `car1` → the object

And in Object-Oriented Programming, objects are built around four major ideas. These are often called the **Four Pillars of OOP** (*You will study these much deeper in the OOP course*).

- **Abstraction:** Abstraction means hiding unnecessary complexity and only showing the important parts.
 - For example, when you drive a car, you use the steering wheel, pedals, brakes. But you do not need to understand the entire engine system just to drive.

- Python classes work similarly. We interact with objects through simple interfaces while hiding internal details.
- **Encapsulation:** Encapsulation means keeping data and methods together inside one unit. A class combines attributes (data) and methods (behavior) into a single structure. This helps protect and organize the data properly.

```
class BankAccount:  
    def __init__(self, balance):  
        self.balance = balance
```

The account balance belongs to the object itself.

- **Inheritance:** Inheritance allows one class to reuse features from another class. Instead of rewriting code repeatedly, we extend existing classes.

```
class Animal:  
    def speak(self):  
        print("Animal sound")  
  
class Dog(Animal):  
    pass
```

Here, `Dog` automatically inherits the `speak()` method from `Animal`.

- **Polymorphism:** Polymorphism means the same method can behave differently depending on the object.

```
class Cat:  
    def sound(self):  
        print("Meow")  
  
class Dog:  
    def sound(self):  
        print("Bark")
```

Both classes use `sound()`, but produce different outputs. Same method name → different behavior.

3. Object-Oriented Programming

Now let's briefly introduce some common OOP concepts you will encounter in future OOP course (*if you decide to take it at all*). For now, we keep everything simple.

Class Methods

Methods are functions inside a class.

```
class Student:
    def greet(self):
        print("Hello student")
```

Objects can use these methods.

```
s1 = Student()
s1.greet()
```

Subclasses

A subclass is a class that inherits from another class.

```
class Person:
    pass

class Teacher(Person):
    pass
```

`Teacher` becomes a subclass of `Person`.

Constructor and Destructor

The constructor initializes the object.

```
def __init__(self):
```

A destructor runs when the object is destroyed.

```
def __del__(self):
```

Example:

```
class Test:
    def __init__(self):
        print("Object created")

    def __del__(self):
        print("Object destroyed")
```

Getter and Setter

These methods control how attributes are accessed or modified.

```
class Person:
    def __init__(self):
        self.__age = 0

    def set_age(self, age):
        self.__age = age

    def get_age(self):
        return self.__age
```

This helps protect internal data.

Duck Typing

Python follows a concept called **Duck Typing**. The idea is simple:

“If it behaves like a duck, then it is a duck.”

Python cares more about behavior than exact type.

```
class Bird:
    def sound(self):
        print("Chirp")

class Human:
    def sound(self):
        print("Hello")

def make_sound(obj):
    obj.sound()

make_sound(Bird())
make_sound(Human())
```

Both objects work because both contain the `sound()` method.

Some Other Concepts

As you continue learning OOP, you will encounter many more ideas:

- Method Overriding
- Method Overloading
- Multiple Inheritance
- Composition
- Operator Overloading
- Abstract Classes
- Interfaces
- Magic Methods (`__str__`, `__len__`, etc.)

But for now, understanding the foundations is enough.

Is treating everything as objects better than using many functions?

In many cases, yes. Because objects help us:

- Organize code better
- Reuse functionality
- Model real-world systems naturally
- Make large programs easier to manage
- Reduce repeated code

Functions are still extremely important. But as programs become larger and more complex, Object-Oriented Programming becomes much more powerful and scalable.

That's why modern software development relies heavily on OOP concepts.

Summary

Classes and Objects are the foundation of Object-Oriented Programming in Python.

A class acts as a blueprint, while objects are the actual instances created from that blueprint.

We also explored the four major pillars of OOP:

- Abstraction
- Encapsulation
- Inheritance
- Polymorphism

Along the way, we introduced several important OOP concepts such as:

- Class methods
- Subclasses
- Constructors and destructors

- Getters and setters
- Duck typing

Most importantly, Object-Oriented Programming allows us to organize programs into structured, reusable, and scalable systems.

Instead of treating code as disconnected functions and variables... we begin thinking in terms of objects that contain both data and behavior.

And this shift in thinking becomes one of the most important transitions in programming.

References

<https://docs.python.org/3/tutorial/classes.html>

<https://www.geeksforgeeks.org/python/python-classes-and-objects/>

Next Section:

 [24. Python: Iterators and Generators](#)